6.4 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-6.1 What values are returned during the following series of stack operations, if executed upon an initially empty stack? push(5), push(3), pop(), push(2), push(8), pop(), pop(), push(9), push(1), pop(), push(7), push(6), pop(), pop(), push(4), pop(), pop().

R-6.2 Suppose an initially empty stack S has executed a total of 25 push operations, 12 top operations, and 10 pop operations, 3 of which raised Empty errors that were caught and ignored. What is the current size of S?

R-6.3 Implement a function with signature transfer(S, T) that transfers all elements from stack S onto stack T, so that the element that starts at the top of S is the first to be inserted onto T, and the element at the bottom of S ends up at the top of T.

R-6.4 Give a recursive method for removing all the elements from a stack.

R-6.5 Implement a function that reverses a list of elements by pushing them onto a stack in one order, and writing them back to the list in reversed order.

R-6.6 Give a precise and complete definition of the concept of matching for grouping symbols in an arithmetic expression. Your definition may be recursive.

R-6.7 What values are returned during the following sequence of queue operations, if executed on an initially empty queue? enqueue(5), enqueue(3), dequeue(), enqueue(2), enqueue(8), dequeue(), dequeue(), enqueue(9), enqueue(1), dequeue(), enqueue(7), enqueue(6), dequeue(), dequeue(), enqueue(4), dequeue(), dequeue().

R-6.8 Suppose an initially empty queue Q has executed a total of 32 enqueue operations, 10 first operations, and 15 dequeue operations, 5 of which raised Empty errors that were caught and ignored. What is the current size of Q?

R-6.9 Had the queue of the previous problem been an instance of ArrayQueue that used an initial array of capacity 30, and had its size never been greater than 30, what would be the final value of the _front instance variable?

R-6.10 Consider what happens if the loop in the ArrayQueue._resize method at lines 53–55 of Code Fragment 6.7 had been implemented as:

```
for k in range(self._size):
  self._data[k] = old[k]                    # rather than old[walk]
```

Give a clear explanation of what could go wrong.

R-6.11 Give a simple adapter that implements our queue ADT while using a collections.deque instance for storage.

R-6.12 What values are returned during the following sequence of deque ADT operations, on initially empty deque? add_first(4), add_last(8), add_last(9), add_first(5), back( ), delete_first( ), delete_last( ), add_last(7), first( ), last( ), add_last(6), delete_first( ), delete_first( ).

R-6.13 Suppose you have a deque D containing the numbers $(1,2,3,4,5,6,7,8)$, in this order. Suppose further that you have an initially empty queue Q. Give a code fragment that uses only D and Q (and no other variables) and results in D storing the elements in the order $(1,2,3,5,4,6,7,8)$.

R-6.14 Repeat the previous problem using the deque D and an initially empty stack S.

Creativity

C-6.15 Suppose Alice has picked three distinct integers and placed them into a stack S in random order. Write a short, straight-line piece of pseudo-code (with no loops or recursion) that uses only one comparison and only one variable x, yet that results in variable x storing the largest of Alice's three integers with probability $2/3$. Argue why your method is correct.

C-6.16 Modify the ArrayStack implementation so that the stack's capacity is limited to maxlen elements, where maxlen is an optional parameter to the constructor (that defaults to None). If push is called when the stack is at full capacity, throw a Full exception (defined similarly to Empty).

C-6.17 In the previous exercise, we assume that the underlying list is initially empty. Redo that exercise, this time preallocating an underlying list with length equal to the stack's maximum capacity.

C-6.18 Show how to use the transfer function, described in Exercise R-6.3, and two temporary stacks, to replace the contents of a given stack S with those same elements, but in reversed order.

C-6.19 In Code Fragment 6.5 we assume that opening tags in HTML have form `<name>`, as with `<li>`. More generally, HTML allows optional attributes to be expressed as part of an opening tag. The general form used is `<name attribute1="value1" attribute2="value2">`; for example, a table can be given a border and additional padding by using an opening tag of `<table border="3" cellpadding="5">`. Modify Code Fragment 6.5 so that it can properly match tags, even when an opening tag may include one or more such attributes.

C-6.20 Describe a nonrecursive algorithm for enumerating all permutations of the numbers $\{1,2,\ldots,n\}$ using an explicit stack.

C-6.21 Show how to use a stack S and a queue Q to generate all possible subsets of an n-element set T nonrecursively.

C-6.22 ***Postfix notation*** is an unambiguous way of writing an arithmetic expression without parentheses. It is defined so that if "(exp_1) **op** (exp_2)" is a normal, fully parenthesized expression whose operation is **op**, the postfix version of this is "$pexp_1$ $pexp_2$ **op**", where $pexp_1$ is the postfix version of exp_1 and $pexp_2$ is the postfix version of exp_2. The postfix version of a single number or variable is just that number or variable. For example, the postfix version of "$((5+2)*(8-3))/4$" is "$5\ 2\ +\ 8\ 3\ -\ *\ 4\ /$". Describe a nonrecursive way of evaluating an expression in postfix notation.

C-6.23 Suppose you have three nonempty stacks R, S, and T. Describe a sequence of operations that results in S storing all elements originally in T below all of S's original elements, with both sets of those elements in their original order. The final configuration for R should be the same as its original configuration. For example, if $R = [1,2,3]$, $S = [4,5]$, and $T = [6,7,8,9]$, the final configuration should have $R = [1,2,3]$ and $S = [6,7,8,9,4,5]$.

C-6.24 Describe how to implement the stack ADT using a single queue as an instance variable, and only constant additional local memory within the method bodies. What is the running time of the push(), pop(), and top() methods for your design?

C-6.25 Describe how to implement the queue ADT using two stacks as instance variables, such that all queue operations execute in amortized $O(1)$ time. Give a formal proof of the amortized bound.

C-6.26 Describe how to implement the double-ended queue ADT using two stacks as instance variables. What are the running times of the methods?

C-6.27 Suppose you have a stack S containing n elements and a queue Q that is initially empty. Describe how you can use Q to scan S to see if it contains a certain element x, with the additional constraint that your algorithm must return the elements back to S in their original order. You may only use S, Q, and a constant number of other variables.

C-6.28 Modify the ArrayQueue implementation so that the queue's capacity is limited to maxlen elements, where maxlen is an optional parameter to the constructor (that defaults to None). If enqueue is called when the queue is at full capacity, throw a Full exception (defined similarly to Empty).

C-6.29 In certain applications of the queue ADT, it is common to repeatedly dequeue an element, process it in some way, and then immediately enqueue the same element. Modify the ArrayQueue implementation to include a rotate( ) method that has semantics identical to the combination, Q.enqueue(Q.dequeue( )). However, your implementation should be more efficient than making two separate calls (for example, because there is no need to modify _size).

C-6.30 Alice has two queues, Q and R, which can store integers. Bob gives Alice 50 odd integers and 50 even integers and insists that she store all 100 integers in Q and R. They then play a game where Bob picks Q or R at random and then applies the round-robin scheduler, described in the chapter, to the chosen queue a random number of times. If the last number to be processed at the end of this game was odd, Bob wins. Otherwise, Alice wins. How can Alice allocate integers to queues to optimize her chances of winning? What is her chance of winning?

C-6.31 Suppose Bob has four cows that he wants to take across a bridge, but only one yoke, which can hold up to two cows, side by side, tied to the yoke. The yoke is too heavy for him to carry across the bridge, but he can tie (and untie) cows to it in no time at all. Of his four cows, Mazie can cross the bridge in 2 minutes, Daisy can cross it in 4 minutes, Crazy can cross it in 10 minutes, and Lazy can cross it in 20 minutes. Of course, when two cows are tied to the yoke, they must go at the speed of the slower cow. Describe how Bob can get all his cows across the bridge in 34 minutes.

Projects

P-6.32 Give a complete ArrayDeque implementation of the double-ended queue ADT as sketched in Section 6.3.2.

P-6.33 Give an array-based implementation of a double-ended queue supporting all of the public behaviors shown in Table 6.4 for the collections.deque class, including use of the maxlen optional parameter. When a length-limited deque is full, provide semantics similar to the collections.deque class, whereby a call to insert an element on one end of a deque causes an element to be lost from the opposite side.

P-6.34 Implement a program that can input an expression in postfix notation (see Exercise C-6.22) and output its value.

P-6.35 The introduction of Section 6.1 notes that stacks are often used to provide "undo" support in applications like a Web browser or text editor. While support for undo can be implemented with an unbounded stack, many applications provide only ***limited*** support for such an undo history, with a fixed-capacity stack. When push is invoked with the stack at full capacity, rather than throwing a Full exception (as described in Exercise C-6.16), a more typical semantic is to accept the pushed element at the top while "leaking" the oldest element from the bottom of the stack to make room.

Give an implementation of such a LeakyStack abstraction, using a circular array with appropriate storage capacity. This class should have a public interface similar to the bounded-capacity stack in Exercise C-6.16, but with the desired leaky semantics when full.

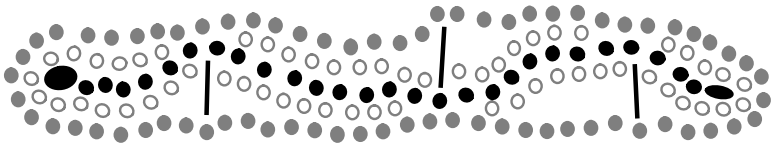
P-6.36 When a share of common stock of some company is sold, the ***capital gain*** (or, sometimes, loss) is the difference between the share's selling price and the price originally paid to buy it. This rule is easy to understand for a single share, but if we sell multiple shares of stock bought over a long period of time, then we must identify the shares actually being sold. A standard accounting principle for identifying which shares of a stock were sold in such a case is to use a FIFO protocol—the shares sold are the ones that have been held the longest (indeed, this is the default method built into several personal finance software packages). For example, suppose we buy 100 shares at $20 each on day 1, 20 shares at $24 on day 2, 200 shares at $36 on day 3, and then sell 150 shares on day 4 at $30 each. Then applying the FIFO protocol means that of the 150 shares sold, 100 were bought on day 1, 20 were bought on day 2, and 30 were bought on day 3. The capital gain in this case would therefore be $100 \cdot 10 + 20 \cdot 6 + 30 \cdot (-6)$, or $940. Write a program that takes as input a sequence of transactions of the form "`buy` x `share(s) at $`y `each`" or "`sell` x `share(s) at $`y `each`," assuming that the transactions occur on consecutive days and the values x and y are integers. Given this input sequence, the output should be the total capital gain (or loss) for the entire sequence, using the FIFO protocol to identify shares.

P-6.37 Design an ADT for a two-color, double-stack ADT that consists of two stacks—one "red" and one "blue"—and has as its operations color-coded versions of the regular stack ADT operations. For example, this ADT should support both a red push operation and a blue push operation. Give an efficient implementation of this ADT using a single array whose capacity is set at some value N that is assumed to always be larger than the sizes of the red and blue stacks combined.

Chapter Notes

We were introduced to the approach of defining data structures first in terms of their ADTs and then in terms of concrete implementations by the classic books by Aho, Hopcroft, and Ullman [5, 6]. Exercises C-6.30, and C-6.31 are similar to interview questions said to be from a well-known software company. For further study of abstract data types, see Liskov and Guttag [71], Cardelli and Wegner [23], or Demurjian [33].

Chapter 7 Linked Lists



Contents

7.1 Singly Linked Lists 256
7.1.1 Implementing a Stack with a Singly Linked List 261
7.1.2 Implementing a Queue with a Singly Linked List 264
7.2 Circularly Linked Lists 266
7.2.1 Round-Robin Schedulers 267
7.2.2 Implementing a Queue with a Circularly Linked List 268
7.3 Doubly Linked Lists 270
7.3.1 Basic Implementation of a Doubly Linked List 273
7.3.2 Implementing a Deque with a Doubly Linked List 275
7.4 The Positional List ADT 277
7.4.1 The Positional List Abstract Data Type 279
7.4.2 Doubly Linked List Implementation 281
7.5 Sorting a Positional List 285
7.6 Case Study: Maintaining Access Frequencies 286
7.6.1 Using a Sorted List 286
7.6.2 Using a List with the Move-to-Front Heuristic 289
7.7 Link-Based vs. Array-Based Sequences 292
7.8 Exercises 294